

Guión del proyecto final (Parte 1)

Metodología de la Programación. Curso 2025/26

Grado en Ingeniería Informática. Grupo F

1 Sobre esta práctica

El objetivo de la práctica final es construir una plataforma de streaming de música (llamada "Spotifry") en el que distintos usuarios podrán guardar y reproducir canciones, crear listas de reproducción, entre otras funcionalidades.

En esta primera parte de la práctica, se desarrollarán las clases esenciales para el desarrollo de la práctica.

Se recuerda que la realización de la práctica final es INDIVIDUAL. Cualquier copia detectada supondrá una calificación de 0 en dicha práctica para todos los estudiantes implicados en la copia. Además, queda expresamente prohibido el uso de herramientas de Inteligencia Artificial generativa para la resolución de la práctica. La autoría de la práctica será verificada mediante entrevistas individuales y/o pruebas prácticas, la no superación de dicha defensa conlleva una calificación de 0 en dicha práctica

Para el desarrollo de la práctica es OBLIGATORIO utilizar compilación separada (con *makefile*), de forma que para cada clase se deberá separar la definición de la misma en un fichero `.h` (o `.h/.hpp`) y los detalles de su implementación en otro fichero `.cpp` (o `.cc`). Recordad que los nombres de los ficheros de las clases deben (en minúscula) coincidir con los nombres de sus clases.

Se valorará una adecuada modularización del código, evitando en la medida de lo posible la duplicación de código innecesario. Además, se valorará la limpieza del código, utilizando nombres de variables y métodos descriptivos y una adecuada indentación del código.

Junto con el enunciado de la práctica se proporciona un ejemplo de programa principal para probar los distintos métodos de las clases. No obstante, la práctica podrá ser verificada con otros programas principales adicionales al proporcionado.

Para la entrega de la práctica final se deberán proporcionar todos los ficheros fuente del proyecto, así como un fichero *makefile* que permita la compilación automática del proyecto en un entorno GNU/Linux.

Recomendaciones:

1. **Lee detenidamente todo este guion antes de programar nada.**
2. Sigue un desarrollo incremental, añadiendo funcionalidades de forma progresiva, una vez se han probado las que se han añadido anteriormente.
3. Desarrolla todos aquellos métodos privados que consideres necesarios para permitir una adecuada modularización del programa, evitando en la medida de lo posible repetir el mismo código en distintas partes del mismo.

2 Clase User

A continuación se proporciona la definición básica de la clase User. Ten en cuenta que Date, Song, Album, Artist y Playlist son clases que aparecen descritas más adelante en este guion.

```
1  enum Gender {FEMALE, MALE, OTHER};
2
3  class User {
4      string name;           // Nombre y apellidos
5      string email;         // Correo electrónico
6      Date birthdate;       // Fecha de nacimiento
7      Gender gender;        // Género
8
9      int max_saved_songs;   // Tamaño del vector "saved_songs"
10     int max_playlists;     // Tamaño del vector "wishlist"
11
12     int num_saved_songs;   // Número actual de canciones en "saved_songs"
13     int num_playlists;     // Número actual de listas de reproducción en "playlists"
14
15     Song** saved_songs;    // La lista de canciones guardadas
16     Playlist** playlists;  // La lista de playlists del usuario
17 };
```

2.1 Constructor de la clase User

Implementar un constructor con parámetros que inicializará los distintos atributos de la clase Usuario.

```
1  User(string name, string email, Date birthdate, Gender gender)
```

Los atributos `max_saved_songs` y `max_playlists` se inicializarán a 4, reservando memoria para su vector correspondiente de acuerdo a este número. Los tamaños de estos vectores nunca deberán ser inferiores a 4.

Al implementar un constructor con parametros será necesario definir el constructor por defecto también (sin ningún parámetro).

2.2 Métodos get y set

Implementar todos los métodos get y set de la clase User que consideres necesarios.

2.3 Método para añadir canciones

```
bool addSong(Song* song_to_add);
```

Implementar un método para añadir una canción a la lista de canciones guardadas del usuario. Se deben cumplir las siguientes condiciones:

1. La canción no está ya en la lista de canciones guardadas.
2. Hay espacio suficiente para añadir la canción. En caso de que no haya espacio suficiente, se extenderá la memoria reservada al doble del valor actual. El valor de `max_saved_songs` nunca deberá ser inferior a 4.
3. Devolverá verdadero o falso dependiendo de si se ha podido añadir la canción.

2.4 Método para añadir una lista de reproducción

```
bool addPlaylist(Playlist* playlist_to_add);
```

Implementar un método para añadir una lista de reproducción a la colección de playlists del usuario. Se deben cumplir las siguientes condiciones:

1. La playlist no está ya en la colección.
2. Hay espacio suficiente para añadir la playlist. En caso de que no haya espacio suficiente, se extenderá la memoria reservada al doble del valor actual. El valor de `max_playlists` nunca deberá ser inferior a 4.
3. Devolverá verdadero o falso dependiendo de si se ha podido añadir la playlist.

2.5 Métodos para eliminar

Crear métodos para eliminar la canción o la playlist dadas de sus correspondientes listas.

```
bool deleteSong(Song* song_to_delete);  
bool deletePlaylist(Playlist* playlist_to_delete);
```

1. Para eliminarlo, el elemento debe existir en la correspondiente lista.
2. No deben quedar huecos en la correspondiente lista. Si el número actual de elementos (`num_saved_songs` o `num_playlists`) es inferior a la mitad del máximo de la correspondiente lista (`max_saved_songs` o `max_playlists`), reducir el tamaño del vector a la mitad del máximo actual, que de nuevo nunca podrá ser inferior a 4.

3. Debe de tenerse en cuenta a la hora de eliminar una playlist si se ha creado por el propio usuario (lista privada) o si es una lista pública de otros usuarios.
4. Devolverá verdadero o falso dependiendo de si se ha podido eliminar el elemento.

2.6 Método para crear una playlist

```
bool createPlaylist(Song** songs, int num_songs, string title);
```

Implementar un método para crear una nueva playlist que contenga las canciones que se pasen en el vector `songs`. El creador de la playlist será el propio usuario y la lista será de tipo PRIVADO.

2.7 Otros métodos

Al usar memoria dinámica es necesario implementar destructor, constructor copia y sobrecarga del operador de asignación.

3 Clase Song

```
1 enum Genre {POP, ROCK, HIPHOP, COUNTRY, JAZZ, BLUES,  
2             ELECTRONIC, REGGAE, CLASSICAL, FOLK, METAL};  
3  
4 class Song {  
5     string title;  
6     Genre genre;  
7     int duration;  
8 };
```

3.1 Constructor de la clase Song

Implementar el constructor de acuerdo al siguiente prototipo:

```
Song(string title, Genre genre, int duration);
```

Al implementar un constructor con parametros será necesario definir el constructor por defecto también (sin ningún parámetro).

3.2 Métodos get y set

Implementar todos los métodos get y set que consideres necesarios.

3.3 Operadores de comparación

Sobrecargar los operadores == y !=. Dos canciones son iguales si coinciden título, género, y duración.

4 Clase Album

```
1 class Album {
2     string title;
3     Artist* artist;
4     Date releaseDate;
5     int num_songs;
6     Song** songs;
7 };
```

4.1 Constructor de la clase Album

Implementar el constructor de acuerdo al siguiente prototipo:

```
Album(string title, Artist* artist, Date releaseDate, Songs** songs, int num_songs);
```

Al implementar un constructor con parametros será necesario definir el constructor por defecto también (sin ningún parámetro).

4.2 Métodos get y set

Implementar todos los métodos get y set que consideres necesarios.

4.3 Otros métodos

Al usar memoria dinámica es necesario implementar destructor, constructor copia y operador de asignación.

4.4 Operador de indexación

Sobrecargar el operador de indexación ([]) para acceder a las canciones de cada album.

5 Clase Artist

```
1 class Artist {
2     string name;
3 };
```

5.1 Constructor de la clase Artist

Implementar el constructor de acuerdo al siguiente prototipo:

```
Artist(string name);
```

Al implementar un constructor con parametros será necesario definir el constructor por defecto también (sin ningún parámetro).

5.2 Métodos get y set

Implementar todos los métodos get y set que consideres necesarios.

6 Clase Playlist

```
1
2 enum PlaylistPrivacy {PUBLIC, PRIVATE};
3
4 class Playlist {
5     string title;
6     User* creator;
7     Date creationDate;
8     int max_songs;
9     int num_songs;
10    Song** songs;
11    PlaylistPrivacy privacy;
12 };
```

6.1 Constructor de la clase Playlist

Implementar el constructor de acuerdo al siguiente prototipo:

```
Playlist(string title, User* creator, Date creationDate, PlaylistPrivacy privacy);
```

La lista de canciones comenzará vacía. Al implementar un constructor con parametros será necesario definir el constructor por defecto también (sin ningún parámetro).

6.2 Métodos get y set

Implementar todos los métodos get y set que consideres necesarios.

6.3 Añadir y eliminar canciones

Implementar métodos para añadir y eliminar canciones de la playlist.

```
bool addSong(Song* song, User* whoAddsIt);  
bool deleteSong(Song* song, User* whoRemovesIt);
```

Se debe usar el mismo comportamiento de memoria dinámica ya descrito (doblar si se llena, reducir si se queda a menos de la mitad de su capacidad). Sólo se pueden añadir o eliminar canciones si quien lo hace (`whoAddsIt` o `whoRemovesIt`, respectivamente) es el creador de la lista.

6.4 Otros métodos

Al usar memoria dinámica es necesario implementar destructor, constructor copia y operador de asignación.

6.5 Operadores de comparación

Sobrecargar los operadores `==` y `!=`. Dos playlist son iguales si coincide el autor y el título.

7 Clase Date

Diseñar e implementar una clase `Date`, que considere día, mes y año, con los métodos y atributos que sean necesarios. Sobrecargar los operadores de comparación (`==`, `!=`, `<`, `>`, `<=` y `>=`).

8 Programa Principal

Junto con este guion se proporciona un ejemplo de programa principal para probar la funcionalidad de las distintas clases desarrolladas en el guion. El software proporcionado debe funcionar con este programa principal sin realizar ningún cambio. Esto no implica que no se deban de realizar pruebas adicionales para comprobar el correcto funcionamiento del programa.